Phase #5 - Back End Unit Testing

CSCI 3060U • Banking System • White Box Test Report

Group Members:

- Aliseena Ahmar (100870078)
- Haider Saleem (100870637)
- Samir Ahmadi (100916280)
- Smit Kalathia (100871659)

Method used for statement coverage	parse_account_line() in read.py
Method used for decision and loop coverage	process_transactions() in backend_processor.py

Two back end methods were selected from the Phase 4 codebase. parse_account_line() was tested for statement coverage because it contains multiple validation decisions and a single successful object-creation path. process_transactions() was tested for decision and loop coverage because it contains a loop over transactions and several decision branches based on the transaction code.

1. Statement coverage analysis

Chosen method: parse_account_line(line, line_number=0) in read.py

Reason for choice: The method contains validation checks for record length, account number, name, status, balance, transaction count, and plan. A small set of tests can execute every important statement in the method at least once.

Test ID	Input	Intention / Description	Expected Result
SC-1	12345John Doe A00100.000000NP	Valid fixed-width account record	Return an Account object with parsed field values
SC-2	12345Short	Line shorter than 40 characters	Raise ValueError for short line
SC-3	12345John Doe X00100.000000NP	Invalid status code	Raise ValueError for invalid status
SC-4	12345John Doe AABCDEFGH0000NP	Balance field not numeric	Raise ValueError for invalid balance
SC-5	12345John Doe A00100.000000XX	Invalid plan code	Raise ValueError for invalid plan

2. Decision and loop coverage analysis

Chosen method: process_transactions(accounts, transactions) in backend_processor.py

Reason for choice: This method contains a for-loop over all transactions and a chain of decisions for each supported transaction code. The selected tests make the loop execute zero times, one time, and more than one time, while also exercising all major decision outcomes.

Test ID	Transactions	Intention / Description	Expected Result

DLC-1	[]	Loop executes 0 times	No balances or plans change
DLC-2	[DEP 12345 50.00]	Deposit branch	Account 12345 balance becomes 150.00
DLC-3	[WDR 54321 20.00]	Withdrawal branch	Account 54321 balance becomes 230.50
DLC-4	[XFR 12345 25.00 54321]	Transfer branch	12345 decreases, 54321 increases
DLC-5	[NEW 11111 0.00 Alice Lee]	Create branch	New account 11111 is added
DLC-6	[DEL 12345 0.00]	Delete branch	Account 12345 is removed
DLC-7	[DSB 54321 0.00]	Disable branch	Account 54321 status becomes D
DLC-8	[CHG 12345 0.00 SP]	Valid change-plan branch	Plan changes from NP to SP
DLC-9	[CHG 12345 0.00 XX]	Invalid change-plan branch	Plan remains NP and no update occurs
DLC-10	[DEP, EOS, WDR]	EOS break branch	Processing stops at EOS
DLC-11	[ABC 12345 10.00]	Unsupported-code else branch	Error is printed and no account changes
DLC-12	[DEP, WDR, CHG]	Loop executes more than 1 time	All three transactions are applied in order

Loop Coverage Explanation

The test cases ensure that the loop in process_transactions() executes zero times (DLC-1), one time (DLC-2 to DLC-11), and more than one time (DLC-12), satisfying loop coverage requirements.

3. Unit test execution results

Command used: python -m unittest discover -s tests -p 'test_*.py' -v

Overall result: 17 tests executed, 17 passed.

Test ID	Method	Result	Observed Outcome
SC-1	parse_account_line()	Pass	Account object returned with correct parsed fields
SC-2	parse_account_line()	Pass	ValueError raised for short line
SC-3	parse_account_line()	Pass	ValueError raised for invalid status
SC-4	parse_account_line()	Pass	ValueError raised for invalid balance field

SC-5	parse_account_line()	Pass	ValueError raised for invalid plan code

Console notes from the decision and loop suite: the program printed two expected error messages during branch testing: Unsupported transaction code: ABC, and Change plan failed: invalid plan 'XX'.

Test ID	Method	Result	Observed Outcome
DLC-1	process_transactions()	Pass	Loop ran 0 times and accounts were unchanged
DLC-2	process_transactions()	Pass	Deposit applied correctly
DLC-3	process_transactions()	Pass	Withdrawal applied correctly
DLC-4	process_transactions()	Pass	Transfer moved funds correctly
DLC-5	process_transactions()	Pass	New account created successfully
DLC-6	process_transactions()	Pass	Account deleted successfully
DLC-7	process_transactions()	Pass	Account disabled successfully
DLC-8	process_transactions()	Pass	Plan changed to SP
DLC-9	process_transactions()	Pass	Invalid plan left account unchanged
DLC-10	process_transactions()	Pass	EOS stopped further processing
DLC-11	process_transactions()	Pass	Unsupported code caused no account changes
DLC-12	process_transactions()	Pass	Multiple transactions applied in order

4. Failures uncovered by testing

Supplementary defect checks were executed after the coverage suites. These checks used the same back end methods but compared the actual behavior against expected banking rules. Three defects were found.

Failure ID	Test Case	Expected Result	Actual Result	Status
F-1	Withdrawal larger than available balance	Transaction rejected and balance remains 100.00	Balance became -50.00	Fail
F-2	Negative deposit amount	Transaction rejected and balance remains 100.00	Balance became 80.00	Fail

F-3	Transfer larger than source balance	Transfer rejected and balances remain unchanged	Source became -400.00; destination became 750.50	Fail

Failures Identified

Three defects were uncovered during additional testing of process_transactions(). First, withdrawals greater than the available balance were still processed, causing the account balance to become negative. Second, negative deposit amounts were accepted, which incorrectly reduced the account balance. Third, transfers larger than the source account balance were processed without validation, causing the source account to become negative while still increasing the destination account balance. These failures show that the back end is missing validation checks for transaction amounts and available funds.

And for the **results summary**, use:

Test Results Summary

A total of 17 unit tests were executed for the two selected methods. All 17 unit tests passed successfully. In addition, supplementary failure-check tests were run to identify possible defects in the transaction processing logic. These additional tests uncovered 3 failures related to missing validation for insufficient funds and negative transaction amounts.

So your final outcome is:

- 17/17 coverage tests passed
- 3 real failures identified

5. Files included

backend/ (contains all Phase 4 back end source files)

tests/test_statement_coverage.py

tests/test_decision_loop_coverage.py

tests/phase5_failure_check.py

results/phase5_unittest_results.txt

results/phase5_failure_results.txt

report/Phase5_Report.docx